

LEARNING RESOURCE GUIDE

ISC2 CCSP: Certified Cloud Security Professional (Aug 2026) | Professional Participant Workbook

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- By the end of this lecture, you will be able to explain the shift-left principle and articulate why moving security earlier in the software development lifecycle reduces both cost and risk.
- By the end of this lecture, you will be able to distinguish between SAST, DAST, and SCA, describe what each tests, and identify the appropriate stage of the CI/CD pipeline where each is applied.
- By the end of this lecture, you will be able to explain the security implications of Infrastructure as Code and describe how IaC scanning tools reduce the risk of configuration-at-scale failures.
- By the end of this lecture, you will be able to describe the security risks of hardcoded secrets and explain how dedicated secrets management solutions address those risks.
- By the end of this lecture, you will be able to explain the security benefits of immutable infrastructure and container image scanning as components of a DevSecOps pipeline.

Introduction

DevOps transformed how organisations build and deploy software by uniting development and operations teams around shared automation, rapid delivery cycles, and continuous feedback. The efficiency gains are real: teams that once released quarterly now release daily, sometimes many times per day. But speed without security discipline creates risk at the same velocity. DevSecOps is the answer — the deliberate integration of security controls into every stage of the development and delivery pipeline so that security is a property of the process rather than a final inspection that the process races past.

The foundational principle of DevSecOps is shift-left: move security activities as early as possible in the software development lifecycle, because the cost and difficulty of fixing a vulnerability grows dramatically with distance from the point of introduction. A vulnerability caught by a pre-commit hook costs minutes to fix. The same vulnerability discovered in a production incident may cost weeks of incident response, regulatory notification, customer notification, legal review, and remediation. The economics of shift-left are not theoretical — they are the empirical finding of every serious study of software defect costs, applied to security specifically.

For the CCSP exam, DevSecOps is examined at the level of concepts and principles. The exam tests whether candidates understand what shift-left means and why it matters, how the major security tool categories — SAST, DAST, and SCA — differ from one another, why Infrastructure as Code introduces configuration risk at scale, how secrets management prevents credential exposure, and why immutable infrastructure is a security property as well as an operational one. This lecture covers all of these, grounding each concept in the kinds of real-world pipeline decisions that cloud security practitioners encounter daily.

Core Concepts Explained

DevSecOps and the Shift-Left Principle

Definition: DevSecOps is the cultural and technical integration of security practices into the DevOps model, making security a shared responsibility across development, operations, and security teams rather than a final gate before release. Shift-left refers to the practice of moving security activities toward the beginning of the software development lifecycle, where defects are cheapest and fastest to correct.

Business relevance: The business case for shift-left is the cost curve of defect remediation. Security research consistently finds that a vulnerability fixed at code review costs orders of magnitude less than the same vulnerability fixed after it reaches production — and exponentially less than one that is exploited. In a DevOps environment where releases happen continuously, the only sustainable way to maintain a strong security posture is to make security checks automatic, fast, and integrated into the tools developers already use.

Cloud Security Practitioner relevance: A Cloud Security Practitioner working in or alongside a DevOps organisation is responsible for ensuring that security controls are built into the CI/CD pipeline — not bolted on as a separate process that teams route around when it slows them down. This means working with development teams to integrate SAST, SCA, and IaC scanning into the pipeline in ways that give developers fast, actionable feedback rather than vague warnings that arrive too late to act on.

Real-world example: A mid-size technology company transitioning from quarterly releases to continuous delivery initially attempted to preserve its existing security model: a manual security review by the security team before each release. Within six months, the security team was reviewing dozens of releases per week and had become the primary bottleneck in the delivery pipeline. Developers began routing around the review by classifying releases as 'minor' to avoid the queue. The company brought in a DevSecOps consultant who helped them automate the most common security checks — SAST in the CI pipeline, SCA in the dependency scanning step, and IaC checks before infrastructure changes were applied. Manual security review was reserved for material architectural changes. Release velocity increased, and the number of security findings caught before production doubled, because automated checks ran on every commit rather than on a sample reviewed by an overloaded team.

Risks or limitations: The primary risk to avoid is treating DevSecOps as a tool purchase rather than a cultural transformation. Buying a SAST tool and adding it to the pipeline without ensuring that developers understand the findings, trust the tool, and have a clear process for triaging and resolving issues produces alert fatigue rather than security improvement. The practice that prevents this is involving developers in the tool selection and tuning process, setting initial thresholds that produce a manageable number of high-confidence findings, and progressively tightening thresholds as the team builds capacity.

SAST, DAST, and SCA: The Testing Triad

Definition: Static Application Security Testing (SAST) analyses source code without executing it, identifying vulnerability patterns at the code level. Dynamic Application Security Testing (DAST) tests a running application from the outside by sending crafted requests, simulating attacker behaviour. Software Composition Analysis (SCA) compares an application's open-source dependencies against vulnerability databases to identify components with known security issues.

Business relevance: Each tool category catches a different class of vulnerability, and none of the three is a substitute for the others. SAST catches code-level issues early. DAST catches runtime configuration and authentication issues that only appear when the application is executing. SCA catches dependency vulnerabilities that are invisible to both SAST and DAST. Together, the three provide coverage across the main vulnerability categories that automated scanning can detect.

Cloud Security Practitioner relevance: A Cloud Security Practitioner designing or reviewing a DevSecOps pipeline ensures that all three testing categories are represented and integrated at the correct pipeline stage. SAST and SCA run in the continuous integration phase on every commit. DAST runs in the test environment after each deployment. The practitioner also ensures that security gates are configured to block promotion of builds that exceed defined severity thresholds, and that findings are routed to the development team with sufficient context to remediate quickly.

Real-world example: A financial services company building a new customer-facing payments API integrated all three testing categories into its CI/CD pipeline. During the first month of operation, SAST using SonarQube identified a SQL injection vulnerability in the account query endpoint — a developer had concatenated user input directly into a database query rather than using parameterised statements. SAST caught this at the commit stage, before the code reached a test environment, and the fix took 20 minutes. Separately, SCA using Snyk flagged that a JSON parsing library in the project's dependency tree had a critical deserialization vulnerability with a published CVE and a CVSS score of 9.8; the library was updated before it reached production. Two weeks after the API was deployed to staging, DAST testing using OWASP ZAP discovered that the password reset endpoint was vulnerable to rate-limiting bypass — an issue that only manifested in the running application and would not have been visible to SAST. Each tool caught something the others could not. The three together gave the team coverage that none individually provided.

Risks or limitations: A common failure is deploying SAST and then ignoring SCA on the assumption that the application's own code is the main risk surface. In modern cloud-native applications, open-source libraries may represent 80 percent or more of the application's executable code, and vulnerability databases like NIST's National Vulnerability Database update daily as new CVEs are published. An application with clean custom code but an unpatched dependency is not a secure application. The practice is to run SCA on every commit and to treat a critical-severity dependency vulnerability with the same urgency as a critical finding in custom code.

Container Security and Image Scanning

Definition: Container security encompasses the practices and controls applied to container images and running containers to ensure they are built from trusted sources, free of known vulnerabilities, and constrained at runtime to the minimum necessary privileges and network access. Container image scanning checks images against vulnerability databases and policy rules before they are deployed.

Business relevance: Containers have become the dominant packaging format for cloud-native applications, which means that vulnerabilities in container images can propagate to every environment where the image is deployed. A single vulnerable base image shared by dozens of microservices is a single point of vulnerability at scale. Image scanning addresses this by making the registry a security gate: images that fail scanning either cannot be pushed to the registry or cannot be deployed to production through admission control policies.

Cloud Security Practitioner relevance: A Cloud Security Practitioner is responsible for ensuring that image scanning is integrated into the CI/CD pipeline and that registry admission controls prevent unscanned or non-compliant images from reaching production. The practitioner also establishes the policy rules — minimum base image standards, prohibited packages, maximum CVSS severity allowed for deployment — and reviews exceptions where teams request to deploy an image that failed scanning.

Real-world example: A platform engineering team at a cloud-native retail company standardised on a minimal Alpine Linux base image for all containerised services after a scanning exercise revealed that their previous Ubuntu-based base image contained 47 packages that were never used by any service and collectively carried 12 known CVEs. After the switch, the average vulnerability count per image dropped from 23 to four. The team integrated Trivy into their CI pipeline to scan every image on build, and configured the container registry to reject images with any critical-severity CVE. Within three months of deploying the policy, one image build was

blocked because the image included an outdated OpenSSL version with a known remote code execution vulnerability — a finding that would previously have reached production undetected. The combination of minimal base images, pipeline scanning, and registry admission control created a defence-in-depth posture for the container supply chain.

Risks or limitations: A frequent mistake is scanning images at build time but not continuously monitoring images already deployed in production. Vulnerability databases update daily, and a container image that was clean on build day may become vulnerable within weeks as new CVEs are published against its components. The practice is to re-scan deployed images on a schedule and to have a defined process for rolling out patched images when a critical vulnerability is published against a component in use.

Infrastructure as Code and IaC Scanning

Definition: Infrastructure as Code is the practice of defining cloud infrastructure — networks, compute instances, databases, storage, IAM roles — in version-controlled code files using tools such as Terraform, AWS CloudFormation, Azure Bicep, or Pulumi. IaC scanning tools analyse these templates for security misconfigurations before they are applied to any real environment.

Business relevance: IaC makes infrastructure reproducible, auditable, and deployable at scale. It also means that a misconfiguration in a template can be applied to hundreds of resources simultaneously with full automation. A storage bucket policy that inadvertently grants public read access, if defined in a Terraform module shared across 30 services, creates 30 publicly readable buckets in a single pipeline run. IaC scanning is the control that catches these misconfigurations before they touch any real environment.

Cloud Security Practitioner relevance: A Cloud Security Practitioner integrates IaC scanning into the CI pipeline as a mandatory check on all infrastructure pull requests. The practitioner defines the policy rules — what constitutes a blocking finding versus a warning — and works with platform teams to remediate findings rather than accumulating exceptions. The practitioner also ensures that IaC scanning covers all template formats in use and that the policy library is kept current with the organisation's compliance requirements.

Real-world example: A healthcare technology company using Terraform to manage its cloud infrastructure integrated Checkov into its CI pipeline as a mandatory check on all Terraform pull requests. In the first 90 days of operation, Checkov blocked 14 pull requests that would have deployed resources with material security issues: three had S3 buckets with public access control lists, four had security groups with inbound rules permitting traffic from 0.0.0.0/0 on sensitive ports, two had RDS instances with encryption disabled, and five had CloudTrail configurations that omitted log file validation. Each finding was caught before it reached any environment. Without IaC scanning, each of these misconfigurations would have been deployed automatically as part of the team's Terraform pipeline — some to production environments containing patient data. The investment in IaC scanning paid for itself on the third blocked deployment.

Risks or limitations: A common limitation is treating IaC scanning as a one-time gate rather than an evolving control. Compliance requirements change, new misconfigurations are documented as the threat landscape evolves, and teams add new resource types that existing policy rules do not cover. The practice is to review and update the policy library on a regular cadence, to subscribe to the scanning tool's vulnerability feed, and to ensure that every new resource type introduced to the Terraform codebase has corresponding policy coverage before it reaches production.

Secrets Management and Immutable Infrastructure

Definition: Secrets management is the practice of storing, rotating, and distributing credentials, API keys, certificates, and other sensitive values through a dedicated, access-controlled system rather than in configuration files, source code, or environment variables that may be exposed. Immutable infrastructure is the

practice of replacing servers and containers rather than modifying them in place, ensuring that every production instance was built from a reviewed, version-controlled template.

Business relevance: Hardcoded secrets in source code are one of the most common causes of cloud breaches. When credentials are committed to a version control repository, they become accessible to everyone with repository access, and repositories are frequently shared more widely than intended — or are accidentally made public. Secrets management eliminates this risk by ensuring that applications retrieve credentials dynamically at runtime from a system that enforces access control and provides full audit logging. Immutable infrastructure eliminates configuration drift and provides forensic clarity: if every server was built from the same audited template, the security posture of each instance is knowable without per-instance inspection.

Cloud Security Practitioner relevance: A Cloud Security Practitioner is responsible for ensuring that a secrets management solution is in place, that all pipeline credentials are retrieved from it rather than stored in CI/CD environment variables, that secret rotation is automated, and that access to secrets is governed by the principle of least privilege through IAM policies. The practitioner also advocates for immutable infrastructure patterns where the team's deployment model allows it, emphasising the security benefit of eliminating the class of vulnerabilities introduced by manual configuration changes applied to long-running servers.

Real-world example: A security audit at a software company discovered that the company's CI/CD pipeline was authenticating to its production AWS account using a long-lived IAM access key stored as a CI system environment variable. The key had been created two years earlier and had never been rotated. It was associated with an IAM user with broad permissions, and three former employees still had access to the CI system settings where the key was visible. The audit also found that three microservices were reading database credentials from configuration files committed to the application repository. The remediation replaced all long-lived keys with short-lived credentials issued by AWS IAM Roles for CI/CD pipelines, moved all application secrets to AWS Secrets Manager with automatic 30-day rotation, and implemented pre-commit hooks using git-secrets to prevent future credential commits. The configuration file credentials were removed from the repository history using git filter-branch. The entire remediation cost less than the estimated liability from a single credential compromise incident based on the volume of customer data in the affected databases.

Risks or limitations: A persistent risk with secrets management is the gap between policy and enforcement: many organisations adopt a secrets manager but continue to allow hardcoded credentials in legacy codebases or CI system settings because migrating them is lower priority than new features. The practice that prevents this is a phased migration plan with a hard cutoff date, pre-commit hooks that block new credential commits, and a periodic audit of the secrets manager access log to identify services that are not using it when they should be.

Practical Applications

The following scenarios illustrate how cloud security practitioners apply DevSecOps principles to real pipeline design, policy enforcement, and incident prevention decisions.

Designing a Security Gate Policy for a Production CI/CD Pipeline

A cloud security practitioner at a SaaS company is tasked with designing the security gate policy for a new CI/CD pipeline that will deploy customer-facing services. The practitioner must balance security rigour with developer velocity — gates that block every medium-severity finding will create a backlog that developers route around; gates that only block critical findings will allow significant vulnerabilities to reach production. The practitioner designs a tiered policy: SAST findings at critical severity block the build immediately with a mandatory developer review before the pipeline can continue; high-severity findings generate a ticket automatically and block

promotion to production but do not block the CI build, giving developers a defined remediation window; medium and low findings are logged and reported in a weekly digest without blocking. SCA follows the same logic applied to CVSS scores. IaC scanning blocks any change that introduces a public access misconfiguration or a security group rule permitting unrestricted inbound access. The policy is documented, reviewed with development leads, and published in the team's internal security standards wiki. The practitioner schedules a quarterly review to reassess thresholds based on actual finding volume and resolution time. This tiered approach is the best practice: security gates that stop work entirely for every finding destroy adoption; gates calibrated to severity and business context create a sustainable security posture.

Responding to a Critical SCA Finding in a Production Dependency

A cloud security practitioner receives an automated alert from the SCA tool at 11 PM: a critical CVE has been published against a JSON parsing library used in seven of the company's production microservices, with a CVSS score of 9.8 and a known public exploit available. The exploit allows remote code execution via a crafted JSON payload in the library's deserialization path. The practitioner's immediate steps follow a defined playbook. First, confirm which services use the affected library version by querying the SCA tool's software bill of materials — all seven services are confirmed. Second, check whether the application's threat model exposes the vulnerable code path to external input in any of the seven services — five do, two do not. Third, escalate the five at-risk services to the on-call development leads with a defined remediation window of four hours for the highest-risk service and 24 hours for the others, based on the blast radius assessment. Fourth, verify that the library vendor has published a patched version — it has, released within hours of the CVE disclosure. Fifth, approve expedited deployment of the patched version through the pipeline after confirming the patch does not introduce breaking API changes. The entire response is completed within six hours of the initial alert. The organisation's response time is possible only because the SCA tool's software bill of materials provided immediate, accurate visibility into the exposure across the service portfolio.

Migrating from Hardcoded Credentials to a Secrets Management Solution

A cloud security practitioner leading a security remediation programme inherits a codebase in which database passwords, API keys for third-party services, and cloud provider credentials are distributed across configuration files, CI/CD environment variables, and a shared password manager that 15 developers have access to. The practitioner designs a phased migration to AWS Secrets Manager. Phase one audits the current credential inventory by scanning the codebase with git-secrets and reviewing CI/CD environment variable configurations, producing a complete list of all secrets and their current storage locations. Phase two migrates the highest-risk secrets — production database credentials and cloud provider access keys — to Secrets Manager within 30 days, updates each affected application to retrieve credentials dynamically at startup, and enables automatic 30-day rotation. Phase three migrates remaining secrets over the following 60 days. Phase four implements pre-commit hooks blocking new credential commits, adds a Secrets Manager access log review to the monthly security checklist, and removes the shared password manager from the approved tools list. At the end of the 90-day programme, every application secret is stored in a system with access control, audit logging, and automatic rotation. The attack surface for credential theft via repository access has been eliminated.

Best Practices

- **Integrate SAST, SCA, and IaC scanning into the CI pipeline as mandatory, automated checks that run on every commit.** Manual security reviews cannot keep pace with continuous delivery pipelines. Automated checks that run on every commit catch findings at the lowest possible cost, give developers immediate feedback while the code is still in context, and do not create a bottleneck that teams are incentivised to

circumvent. Configure each tool to produce actionable output — specific file, line, and remediation guidance — rather than generic warnings that require additional research to act on.

- **Apply DAST in the test environment after every deployment, not only at scheduled intervals.** DAST finds vulnerabilities that only manifest in a running application — authentication flaws, session management issues, business logic vulnerabilities, and runtime configuration errors that SAST and SCA cannot see. Limiting DAST to periodic scheduled scans means that new vulnerabilities introduced between scans may reach production. Integrating DAST as a post-deployment step in the staging pipeline ensures coverage without requiring a separate scheduling process.
- **Store all secrets in a dedicated secrets management solution with automatic rotation and least-privilege access policies.** Hardcoded credentials, CI/CD environment variables containing long-lived keys, and shared password managers are each a single point of credential exposure. A dedicated secrets management solution — AWS Secrets Manager, HashiCorp Vault, Azure Key Vault — provides encryption at rest, access control enforced by IAM, full audit logging of every secret retrieval, and automatic rotation that eliminates the risk of long-lived, never-rotated credentials accumulating in the environment.
- **Configure container registry admission controls to block images that fail scanning before they can be deployed.** Scanning images in the CI pipeline is necessary but insufficient if images can be deployed directly from an engineer's workstation or from outside the automated pipeline. Registry admission controls — Kubernetes admission webhooks, AWS ECR image scanning requirements, or equivalent — enforce the policy at the deployment layer, ensuring that no image reaches production without passing the organisation's scanning and policy criteria regardless of how it was built or pushed.
- **Implement pre-commit hooks to prevent secret commits and treat any credential discovered in version control as compromised immediately.** The moment a credential is committed to a version control repository, it must be treated as compromised regardless of whether the repository is private or immediately accessible to a limited audience. Repositories get shared, forked, or accidentally made public. The correct response to a discovered repository credential is immediate rotation, audit log review for use of the exposed credential, and assessment of what data the credential could have accessed. Pre-commit hooks using tools such as git-secrets or Detect-secrets prevent the problem at the source.

Common Mistakes and Misconceptions

Misconception: Adding a SAST tool to the CI pipeline and considering the security testing programme complete.

Correct approach: SAST covers only code-level vulnerabilities in custom code. SCA is required to address the vulnerability surface of open-source dependencies, which may represent the majority of executable code in a modern application. DAST is required to catch runtime and configuration vulnerabilities that do not appear in static analysis. All three categories are necessary; none is a substitute for the others.

Misconception: Storing pipeline credentials as CI/CD environment variables and rotating them manually on an ad hoc basis.

Correct approach: Long-lived credentials stored in CI/CD systems are accessible to all users with settings access, are frequently not rotated, and accumulate over time as team membership changes. The correct approach is short-lived credentials issued through IAM role assumption or a secrets management solution, with automatic rotation eliminating the dependency on manual processes and reducing the blast radius of any credential exposure.

Misconception: Treating IaC scanning as a one-time setup rather than an evolving control that requires regular policy updates.

Correct approach: New resource types, new misconfiguration patterns, and changing compliance requirements continuously expand the scope of what IaC scanning should check. A policy library that was comprehensive at implementation will have gaps within months. Review and update IaC scanning policies on a quarterly cycle, subscribe to the scanning tool's policy update feed, and ensure that every new resource type added to the infrastructure codebase has corresponding policy coverage before reaching production.

Misconception: Scanning container images only at build time and assuming deployed images remain secure indefinitely.

Correct approach: Vulnerability databases update daily. A container image that was clean on the day it was built may contain one or more newly published critical CVEs within weeks. Implement continuous scanning of deployed images on a scheduled basis and define a process for rolling out patched images when critical vulnerabilities are published against components in use in production.

table: DevSecOps Pipeline Security Control Reference

This table maps each major DevSecOps security control to its pipeline stage, the tool categories that implement it, and the failure class it addresses. Use it to verify that a pipeline design provides coverage across all major risk categories.

Pipeline Stage	Control	Tool Category / Examples	Failure Class Addressed
Developer workstation	Pre-commit hooks for secret detection	git-secrets, Detect-secrets	Credential exposure via version control
CI — code commit	Static Application Security Testing (SAST)	SonarQube, Checkmarx, Semgrep	Code-level vulnerabilities in custom code
CI — code commit	Software Composition Analysis (SCA)	Snyk, Dependabot, OWASP Dependency-Check	Known CVEs in open-source dependencies
CI — image build	Container image scanning	Trivy, Anchore, Clair	Vulnerable packages in container base images
CI — infrastructure change	Infrastructure as Code scanning	Checkov, tfsec, Terrascan	Cloud misconfiguration deployed at scale
CD — staging deployment	Dynamic Application Security Testing (DAST)	OWASP ZAP, Burp Suite	Runtime and authentication vulnerabilities
Runtime — all environments	Secrets management	AWS Secrets Manager, HashiCorp Vault, Azure Key Vault	Credential exposure via configuration or code
Registry	Admission control	ECR scanning policies, Kubernetes admission webhooks	Unscanned or non-compliant images reaching production
Production	Continuous image re-scanning	Scheduled Trivy / Anchore scans	New CVEs published against deployed components

How to use this: Security gates should block promotion at the build or staging stage for critical-severity findings. Medium and low findings should be tracked as remediation tickets with defined SLAs rather than blocking daily delivery.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

Q1. A developer at a cloud-native company commits code containing a hardcoded database password to the application repository. The repository is private. What is the correct security response?

- A) A. No action is required because the repository is private and the credential is not publicly exposed.
- B) B. Document the finding and rotate the credential at the next scheduled maintenance window.
- C) C. Treat the credential as compromised immediately, rotate it, review audit logs for unauthorised use, and implement pre-commit hooks to prevent recurrence.
- D) D. Remove the credential from the current commit and monitor repository access logs for 30 days.

Answer: C. Treat the credential as compromised immediately, rotate it, review audit logs for unauthorised use, and implement pre-commit hooks to prevent recurrence.

Q2. Which of the following correctly distinguishes SAST from DAST?

- A) A. SAST tests a running application by sending crafted HTTP requests; DAST analyses source code for vulnerability patterns.
- B) B. SAST analyses source code without executing it; DAST tests a running application by probing it from the outside.
- C) C. SAST and DAST test the same vulnerability classes and are interchangeable; the choice depends on team preference.
- D) D. SAST is used only in the deployment stage; DAST is used only during code review.

Answer: B. SAST analyses source code without executing it; DAST tests a running application by probing it from the outside.

Q3. A security team integrates SCA into their CI pipeline. The SCA tool reports that a JSON parsing library used by the application has a critical CVE with a CVSS score of 9.8. The library vendor has published a patched version. What is the correct immediate action?

- A) A. Disable the SCA tool until the team has time to evaluate the finding.
- B) B. Accept the risk and continue deployment because the application's own code is not affected.
- C) C. Update the dependency to the patched version, verify the update does not break the application, and deploy through the normal pipeline.
- D) D. Freeze all deployments for 30 days pending a full security review of all dependencies.

Answer: C. Update the dependency to the patched version, verify the update does not break the application, and deploy through the normal pipeline.

Q4. What is the primary security advantage of immutable infrastructure over traditional mutable server management?

- A) A. Immutable infrastructure eliminates the need for container image scanning.
- B) B. Immutable infrastructure prevents configuration drift and ensures every production instance was built from an audited, version-controlled template.
- C) C. Immutable infrastructure automatically rotates secrets and credentials on every deployment.
- D) D. Immutable infrastructure provides stronger network isolation than traditional virtualisation.

Answer: B. Immutable infrastructure prevents configuration drift and ensures every production instance was built from an audited, version-controlled template.

Q5. A platform engineering team scans all container images at build time and blocks deployment of any image with a critical CVE. Three months after implementing this control, a critical CVE is published against the

OpenSSL version included in images that were clean when built and are currently running in production. What is the correct response?

- A) A. No action is required because the images passed scanning at build time.
- B) B. Re-scan deployed images to identify affected instances, build and deploy patched images within the defined remediation SLA for critical vulnerabilities, and implement continuous scanning of deployed images to prevent future gaps.
- C) C. Disable the affected services until the CVE is resolved by the OpenSSL maintainers.
- D) D. Issue a waiver for the existing images and apply the new scanning rule only to future builds.

Answer: B. Re-scan deployed images to identify affected instances, build and deploy patched images within the defined remediation SLA for critical vulnerabilities, and implement continuous scanning of deployed images to prevent future gaps.

Reflection Questions

1. Consider the CI/CD pipeline for a system you work with. At which stages are security checks currently automated? Where does security review still depend on manual effort that could be automated or that creates a bottleneck?
2. Think about how credentials are managed in your current or recent working environment. Are any stored in configuration files, repository environment variables, or shared password managers? What would the impact be if those credentials were exposed tomorrow?
3. If SAST, SCA, and DAST are all required for a complete testing posture, which of the three is most likely to be missing in a DevOps pipeline you are familiar with, and what class of vulnerability does that gap leave uncovered?
4. Consider the concept of immutable infrastructure. What changes to your team's current deployment and patching processes would be required to adopt an immutable infrastructure model? What security benefits would those changes deliver?
5. How would your organisation respond if a critical CVE were published tonight against a library used by five production services? Is there a documented playbook? Does the SCA tool provide immediate visibility into which services are affected?

Key Takeaways

- Shift-left means moving security checks earlier in the development lifecycle where fixing issues is fastest and cheapest. The cost of fixing a vulnerability grows dramatically with its distance from the point of introduction.
- SAST, DAST, and SCA address different vulnerability classes and must all be present in a complete DevSecOps pipeline. None is a substitute for the others.
- SCA is essential in cloud-native applications because open-source dependencies may represent the majority of executable code, and vulnerability databases update daily.
- Hardcoded credentials in source code must be treated as compromised immediately regardless of repository visibility. The correct solution is a dedicated secrets management system with automatic rotation.
- IaC scanning catches configuration mistakes before they are applied to real environments. Without it, a single template misconfiguration can deploy insecure resources at scale across hundreds of services.
- Container image scanning at build time must be complemented by continuous scanning of deployed images because new CVEs are published after images are already running in production.

- Security gates in the CI/CD pipeline are the mechanism that gives security teams programmatic control over what reaches production without requiring manual review of every build.
- Immutable infrastructure eliminates configuration drift and ensures that every production instance's security posture matches its audited, version-controlled template.

Recommended Next Actions

Put this lecture to work within the next seven days:

6. Audit the CI/CD pipeline for one application this week and identify which of the three testing categories — SAST, SCA, and DAST — are present and at which pipeline stages. Document any gaps and prioritise integrating the missing category that addresses the highest-risk vulnerability class for that application.
7. Conduct a credential audit for one application or pipeline this month: scan the codebase with git-secrets or a similar tool, review CI/CD environment variable configurations, and list all credentials not yet stored in a secrets management solution. Migrate the highest-risk credentials to a dedicated solution within 30 days.
8. Review the container image scanning policy for one application environment this week: confirm that scanning is integrated at the build stage, that the registry has admission controls preventing unscanned images from deploying, and that a process exists for re-scanning images already running in production when new critical CVEs are published.
9. Identify one IaC codebase in your environment and run an IaC scanning tool against it this week. Review the findings, categorise them by severity, and create remediation tickets for any critical or high-severity misconfigurations with a defined resolution deadline.